2. Manual de la librería Python — LibPhithon

2.1. Descripción

`LibPhithon` es una librería **cliente ADS** escrita en Python puro (sin dependencias externas; solo la biblioteca estándar). Permite conectar con un dispositivo ADS (real o el simulador Flutter) y leer/escribir variables PLC, gestionar handles y suscribir notificaciones.

2.2. Instalación

```
cd LibPhithon
pip install -e .
```

No requiere paquetes de terceros. Requiere Python ≥ 3.8.

2.3. Uso rápido

```
from ads import AdsClient

with AdsClient(ams_net_id="127.0.0.1.1.1", ams_port=851,
               ip_address="127.0.0.1", tcp_port=48898) as c:
    estado, estado_dispositivo = c.read_state()
    contador = c.read_by_name("MAIN.nCounter", "UINT")
    c.write_by_name("MAIN.bStart", True, "BOOL")
    temperatura = c.read_by_name("MAIN.fTemp", "REAL")
```

2.4. Referencia de la API

class AmsNetId(netid=None)

Identificador de red AMS de 6 bytes, escrito como `a.b.c.d.e.f`.

Método	Qué hace
`__init__(netid=None)`	Construye a partir de una cadena `"a.b.c.d.e.f"`, de `bytes` (6) o de otro `AmsNetId`.
`to_bytes()` / `__bytes__()`	Devuelve los 6 bytes.
`__str__()`	Devuelve la representación `"a.b.c.d.e.f"`.
`is_empty()`	`True` si todos los bytes son 0.

class AmsAddr(netid=None, port=0)

Dirección AMS (NetId + puerto).

class AdsError(code=..., message=None)

Excepción del protocolo. Atributo `.code` = código de error (uint32). El mensaje incluye la descripción del código.

class SymbolInfo

Resultado de consultar la información de un símbolo. Atributos: `index_group`, `index_offset`, `size`, `data_type`, `flags`, `name`, `type_name`, `comment`.

class NotificationSample

Muestra entregada por una notificación. Atributos: `handle`, `timestamp` (FILETIME Windows) y `data` (bytes).

class AdsClient(...)

Cliente principal. Constructor:

```
AdsClient(ams_net_id="127.0.0.1.1.1", ams_port=851,
          ip_address="127.0.0.1", tcp_port=48898,
          timeout=5.0, local_port=0)
```

Parámetro	Qué hace
`ams_net_id`	NetId AMS del destino.
`ams_port`	Puerto AMS del destino (851 = PLC runtime).
`ip_address`	Host al que se conecta el socket TCP.
`tcp_port`	Puerto TCP de AMS/TCP (48898).
`timeout`	Timeout de cada transacción, en segundos.
`local_port`	Puerto AMS local (0 = aleatorio).

Métodos:

Método	Qué hace
`open()`	Abre el socket TCP y arranca el hilo lector de respuestas/notificaciones.
`close()`	Cierra el socket y despierta las transacciones pendientes.
`__enter__()` / `__exit__()`	Soporte de contexto (`with AdsClient(...) as c`).
`read_device_info()`	Envía `READ_DEVICE_INFO` y devuelve `(major, minor, build, name)`.
`read_state()`	Envía `READ_STATE` y devuelve `(ads_state, device_state)`.
`write_control(state, device_state=0,`	Envía `WRITE_CONTROL` para cambiar el

`data=b"")`	estado del dispositivo.
`read_index(index_group, index_offset, size)`	Lectura directa por índice/offset (imagen de proceso, tabla, ...).
`write_index(index_group, index_offset, data)`	Escritura directa por índice/offset.
`read_write(index_group, index_offset, read_len, write_data)`	Comando `READ_WRITE` (0x0009): escribe y lee en una transacción.
`read_symbol_info(name)`	Consulta `SYM_INFOBYNAME` (0xF006) y devuelve `SymbolInfo`.
`get_handle(name)`	Obtiene el handle de un símbolo (`SYM_HNDBYNAME` 0xF003).
`release_handle(handle)`	Libera un handle (`SYM_RELEASEHND` 0xF014).
`read_by_handle(handle, size)`	Lee el valor por handle (`SYM_VALBYHND` 0xF004).
`write_by_handle(handle, data)`	Escribe el valor por handle.
`read_by_name(name, plc_datatype=None)`	Lee por nombre (`SYM_VALBYNAME` 0xF005). Si se da `plc_datatype`, devuelve el valor Python descodificado; si no, devuelve bytes.
`write_by_name(name, value, plc_datatype=None)`	Escribe por nombre. `value` puede ser bytes o un valor Python con `plc_datatype`.
`sum_read(entries)`	Lectura múltiple (`SUMUP_READ` 0xF080). `entries` = lista de `(index_group, index_offset, length)`. Devuelve lista de bytes.
`sum_write(entries)`	Escritura múltiple (`SUMUP_WRITE` 0xF081). `entries` = lista de `(index_group, index_offset, data)`.
`add_device_notification(index_group, index_offset, cb_length, trans_mode=4, max_delay=0, cycle_time=1000000, callback=None)`	Suscribe una notificación y devuelve su handle. `callback` recibe un `NotificationSample`.
`del_device_notification(handle)`	Cancela una notificación.

Módulo ads.datatypes

Función	Qué hace
`size_of(plc_type)`	Devuelve el tamaño en bytes del valor codificado.
`encode_value(value, plc_type)`	Codifica un valor Python a bytes.
`decode_value(data, plc_type)`	Descodifica bytes a un valor Python.

2.5. Tipos PLC soportados

`BOOL`, `BYTE`, `SINT`, `USINT`, `INT`, `UINT`, `WORD`, `DINT`, `UDINT`, `DWORD`,

`REAL`, `TIME`, `TOD`, `DT`, `DATE`, `LINT`, `ULINT`, `LREAL`, `STRING(n)`,

`WSTRING(n)` y `ARRAY[a..b] OF T`.

2.6. Estructura de archivos

Archivo	Qué hace
`ads/constants.py`	Comandos, index groups, estados, puertos y códigos de error.
`ads/errors.py`	`AdsError` y tabla de mensajes de error.
`ads/ams_netid.py`	Clase `AmsNetId`.
`ads/ams_addr.py`	Clase `AmsAddr`.
`ads/packet.py`	Encode/decode del frame AMS/TCP y de la cabecera AMS.
`ads/datatypes.py`	Codificación/decodificación de tipos PLC.
`ads/client.py`	`AdsClient` (conexión, transacciones, notificaciones).

2.7. Pruebas

```
cd LibPhithon
python -m unittest discover -s tests -v
```

2.8. Leer/escribir estructuras completas

Para leer o escribir una **estructura completa** (todos sus miembros de una vez),

define su disposición con `StructMember` y `StructDefinition`:

```
from ads import AdsClient, StructMember, StructDefinition

posicion = StructDefinition([
    StructMember("X", "LREAL", 0),      # offset 0 bytes
    StructMember("Y", "LREAL", 8),      # offset 8 bytes
    StructMember("Activo", "BOOL", 16), # offset 16 bytes
], total_size=24)

with AdsClient(ams_net_id="127.0.0.1.1.1", ams_port=851,
               ip_address="127.0.0.1", tcp_port=48898) as c:
    # Leer toda la estructura de golpe (una sola lectura)
    valores = c.read_struct("GVL.Posicion", posicion)
    # -> {"X": 1.0, "Y": 2.0, "Activo": True}

    # Escribir toda la estructura de golpe (una sola escritura)
    c.write_struct("GVL.Posicion", posicion,
                   {"X": 3.0, "Y": 4.0, "Activo": False})
```

Elemento	Qué hace
`StructMember(nombre, tipo_plc, offset_bytes)`	Define un miembro (nombre, tipo y offset en bytes).
`StructDefinition(miembros, total_size=None)`	Define la estructura. Sin `total_size`, se calcula como `max(offset + tamaño)`.
`read_struct(nombre, def)`	Lee la estructura y devuelve `{miembro:`

	valor}.
write_struct(nombre, def, valores)	Escribe la estructura desde {miembro: valor}.

> El `offset` de cada miembro es su **offset en bytes** dentro de la estructura.

> En el `.tpy` de TwinCAT viene como `BitOffs`: divídelo entre 8.